AMERICAN UNIVERSITY OF BEIRUT

MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

EECE 334 PROJECT

Phase 1: Grammar, FIRST/FOLLOW Sets, Parsing Table, Lexer & Parser

Domain-Specific Language for Multi-Agent AI Workflows

John Kofdarelli - Alex Chbib

Contents

1.Tokens

Token Name	Description / Examples
AGENT, TOOL, TASK, ACTION, SYSTEM, FOR, IN, IF, RUN	Reserved keywords of the DSL
TRUE, FALSE	Boolean literals
STRING_TYPE, INT_TYPE, BOOL_TYPE, LIST_TYPE	Type keywords: string int bool list
ID	Identifier: starts with a letter or _, followed by letters/digits/_
INT_LIT	Integer literal: one or more digits
STRING_LIT	Double-quoted string literal
ARROW (->)	Return-type separator in task declarations
EQEQ, NEQ, LT, GT, LEQ, GEQ	Comparison operators: == != < > <= >=
EQ (=)	Assignment operator
PLUS, MINUS, MULT, DIV	Arithmetic operators: + - * /
LBRACE, RBRACE ({ })	Block delimiters
LPAREN, RPAREN (( ))	Parentheses
LBRACKET, RBRACKET ([ ])	List literal delimiters
COMMA, COLON, DOT (, : .)	Punctuation

2.Nonterminals

Program, AgentList, AgentDef, AgentBody, AgentItem, ToolDecl, TaskDecl, ParamList, ParamListTail, Param, ReturnSpec, ActionList, ActionStmt, ActionArgs, ActionArgsTail, SystemBlock, StmtList, Stmt, ForStmt, IfStmt, Condition, CompOp, Expr, ArgList, ArgTail, ArithExpr, ArithTail, Term, TermTail, Factor, ListLit, ListItems, ListItemsTail, Arg, Type

3.Grammar Rules

Full grammar:

1. Program → AgentList SystemBlock

Agent Definitions:

2. AgentList → AgentDef AgentList | ε
3. AgentDef → agent ID { AgentBody }
4. AgentBody → AgentItem AgentBody | ε
5. AgentItem → ToolDecl | TaskDecl
6. ToolDecl → tool ID

Task Declarations:

7. TaskDecl → task ID (ParamList) ReturnSpec { ActionList }
8. ParamList → Param ParamListTail | ε
9. ParamListTail → , Param ParamListTail | ε
10. Param → Type ID
11. ReturnSpec → -> Type ID | ε

Actions:

12. ActionList → ActionStmt ActionList | ε
13. ActionStmt → action : ID (ActionArgs)
14. ActionArgs → Arg ActionArgsTail | ε
15. ActionArgsTail → , Arg ActionArgsTail | ε

System Workflow:

16. SystemBlock → system { StmtList }
17. StmtList → Stmt StmtList | ε

Statements:

18. Stmt → Type ID = Expr | ID = Expr | ForStmt | IfStmt
19. ForStmt → for ID in ID { StmtList }
20. IfStmt → if Condition { StmtList }
21. Condition → ID CompOp Arg
22. CompOp → == | != | < | > | <= | >=

Expressions:

23. Expr → run ID . ID (ArgList) | ArithExpr
24. ArgList → Arg ArgTail | ε
25. ArgTail → , Arg ArgTail | ε
26. ArithExpr → Term ArithTail
27. ArithTail → + Term ArithTail | - Term ArithTail | ε

28. Term → Factor TermTail
29. TermTail → * Factor TermTail | / Factor TermTail | ε
30. Factor → INT_LIT | STRING_LIT | true | false| ID| (ArithExpr)| ListLit
31. ListLit → [ListItems]
32. ListItems → Arg ListItemsTail | ε
33. ListItemsTail → , Arg ListItemsTail | ε

Base:

34. Arg → STRING_LIT| INT_LIT| ID| true| false
35. Type → string| int| bool| list

4.First

Non-Terminal	FIRST(.)
Type	{ string, int, bool, list }
Arg	{ STRING_LIT, INT_LIT, ID, true, false }
Factor	{ INT_LIT, STRING_LIT, true, false, ID, (, [}
TermTail	{ *, /, ε }
Term	{ INT_LIT, STRING_LIT, true, false, ID, (, [}
ArithTail	{ +, -, ε }
ArithExpr	{ INT_LIT, STRING_LIT, true, false, ID, (, [}
ArgTail	{ ,, ε }
ArgList	{ STRING_LIT, INT_LIT, ID, true, false, ε }
ListItemsTail	{ ,, ε }
ListItems	{ STRING_LIT, INT_LIT, ID, true, false, ε }
ListLit	{ [}
Expr	{ run, INT_LIT, STRING_LIT, true, false, ID, (, [}
CompOp	{ ==, !=, <, >, <=, >= }
Condition	{ ID }
Param	{ string, int, bool, list }
ParamListTail	{ ,, ε }
ParamList	{ string, int, bool, list, ε }
ReturnSpec	{ ->, ε }
ActionArgsTail	{ ,, ε }
ActionArgs	{ STRING_LIT, INT_LIT, ID, true, false, ε }
ActionStmt	{ action }
ActionList	{ action, ε }
ToolDecl	{ tool }
TaskDecl	{ task }
AgentItem	{ tool, task }
AgentBody	{ tool, task, ε }
AgentDef	{ agent }
AgentList	{ agent, ε }
Stmt	{ string, int, bool, list, ID, for, if }
StmtList	{ string, int, bool, list, ID, for, if, ε }
ForStmt	{ for }
IfStmt	{ if }
SystemBlock	{ system }
Program	{ agent, system }

5.Follow

Non-Terminal	FOLLOW(.)
Program	{ $ }
AgentList	{ system }
AgentDef	{ agent, system }
AgentBody	{ }
AgentItem	{ tool, task, } }
ToolDecl	{ tool, task, } }
TaskDecl	{ tool, task, } }
ParamList	{) }
ParamListTail	{) }
Param	{ , ,) }
ReturnSpec	{ { }
ActionList	{ } }
ActionStmt	{ action, } }
ActionArgs	{) }
ActionArgsTail	{) }
SystemBlock	{ $ }
StmtList	{ } }
Stmt	{ string, int, bool, list, ID, for, if, } }
ForStmt	{ string, int, bool, list, ID, for, if, } }
IfStmt	{ string, int, bool, list, ID, for, if, } }
Condition	{ { }
CompOp	{ STRING_LIT, INT_LIT, ID, true, false }
Expr	{ string, int, bool, list, ID, for, if, } }
ArithExpr	{ string, int, bool, list, ID, for, if, },) }
ArithTail	{ string, int, bool, list, ID, for, if, },) }
Term	{ +, -, string, int, bool, list, ID, for, if, },) }
TermTail	{ +, -, string, int, bool, list, ID, for, if, },) }
Factor	{ *, /, +, -, string, int, bool, list, ID, for, if, },) }
ListLit	{ *, /, +, -, string, int, bool, list, ID, for, if, },) }
ListItems	{] }
ListItemsTail	{] }
ArgList	{) }
ArgTail	{) }
Arg	{ , ,),], { }
Type	{ ID }

6.Parse Table

First we divided (a → b|c into a → b and a → c) so we know precise part of rules that derives.

Then we numbered the rules so we can better fit them in the table (R1 = rule 1 and etc..)

(R1) Program → AgentList SystemBlock

Agent Definitions

(R2) AgentList → AgentDef AgentList

(R3) AgentList → ε

(R4) AgentDef → agent ID { AgentBody }

(R5) AgentBody → AgentItem AgentBody

(R6) AgentBody → ε

(R7) AgentItem → ToolDecl

(R8) AgentItem → TaskDecl

(R9) ToolDecl → tool ID

Task Declarations

(R10) TaskDecl → task ID (ParamList) ReturnSpec { ActionList }

(R11) ParamList → Param ParamListTail

(R12) ParamList → ε

(R13) ParamListTail → , Param ParamListTail

(R14) ParamListTail → ε

(R15) Param → Type ID

(R16) ReturnSpec → -> Type ID

(R17) ReturnSpec → ε

Actions

(R18) ActionList → ActionStmt ActionList

(R19) ActionList → ε

(R20) ActionStmt → action : ID (ActionArgs)

(R21) ActionArgs → Arg ActionArgsTail

(R22) ActionArgs → ε

(R23) ActionArgsTail → , Arg ActionArgsTail

(R24) ActionArgsTail → ε

System Workflow

(R25) SystemBlock → system { StmtList }

(R26) StmtList → Stmt StmtList

(R27) StmtList → ε

Statements

(R28) Stmt → Type ID = Expr (variable declaration)

(R29) Stmt → ID = Expr (assignment)

(R30) Stmt → ForStmt

(R31) Stmt → IfStmt

(R32) ForStmt → for ID in ID { StmtList }

(R33) IfStmt → if Condition { StmtList }

(R34) Condition → ID CompOp Arg

(R35) CompOp → ==

(R36) CompOp → !=

(R37) CompOp → <

(R38) CompOp → >

(R39) CompOp → <=

(R40) CompOp → >=

Expressions

(R41) Expr → run ID . ID (ArgList) (run expression)

(R42) Expr → ArithExpr

(R43) ArgList → Arg ArgTail

(R44) ArgList → ε

(R45) ArgTail → , Arg ArgTail

(R46) ArgTail → ε

(R47) ArithExpr → Term ArithTail

(R48) ArithTail → + Term ArithTail

(R49) ArithTail → - Term ArithTail

(R50) ArithTail → ε

(R51) Term → Factor TermTail

(R52) TermTail → * Factor TermTail

(R53) TermTail → / Factor TermTail

(R54) TermTail → ε

(R55) Factor → INT_LIT

(R56) Factor → STRING_LIT

(R57) Factor → true

(R58) Factor → false

(R59) Factor → ID

(R60) Factor → (ArithExpr)

(R61) Factor → ListLit

(R62) ListLit → [ListItems]

(R63) ListItems → Arg ListItemsTail

(R64) ListItems → ε

(R65) ListItemsTail → , Arg ListItemsTail

(R66) ListItemsTail → ε

Bases

(R67) Arg → STRING_LIT

(R68) Arg → INT_LIT

(R69) Arg → ID

(R70) Arg → true

(R71) Arg → false

(R72) Type → string

(R73) Type → int

(R74) Type → bool

(R75) Type → list

The table:

We split the table into 2 parts so they can fit (we couldn't fit all columns next to each other)

Table Part A: Keywords and Type Tokens

Non-Terminal	agent	tool	task	action	system	for	in	if	run	true	false	str	int	bool	list	ID	INT_LIT	STR_LIT
Program	R1				R1													
AgentList	R2				R3													
AgentDef	R4																	
AgentBody		R5	R5															
AgentItem		R7	R8															
ToolDecl		R9																
TaskDecl			R10															
ParamList												R11	R11	R11	R11			
ParamListTail																		
Param												R15	R15	R15	R15			
ReturnSpec																		
ActionList				R18														
ActionStmt				R20														
ActionArgs										R21	R21					R21	R21	R21
ActionArgsTail																		
SystemBlock					R25													
StmtList						R26		R26				R26	R26	R26	R26	R26		
Stmt						R30		R31				R28	R28	R28	R28	R29		
ForStmt						R32												
IfStmt								R33										
Condition																R34		
CompOp																		
Expr									R41	R42	R42					R42	R42	R42
ArgList										R43	R43					R43	R43	R43
ArgTail																		
ArithExpr										R47	R47					R47	R47	R47
ArithTail						R50		R50				R50	R50	R50	R50	R50		
Term										R51	R51					R51	R51	R51
TermTail						R54		R54				R54	R54	R54	R54	R54		
Factor										R57	R58					R59	R55	R56
ListLit																		
ListItems										R63	R63					R63	R63	R63
ListItemsTail																		
Arg										R70	R71					R69	R68	R67
Type												R72	R73	R74	R75			

Table Part B:Operators and Delimiters

Non-Terminal	->	{	}	(	)	[	]	,	==	!=	<	>	<=	>=	=	$
Program																
AgentList																
AgentDef																
AgentBody			R6													
AgentItem																
ToolDecl																
TaskDecl																
ParamList					R12											
ParamListTail					R14			R13								
Param																
ReturnSpec	R16	R17														
ActionList			R19													
ActionStmt																
ActionArgs					R22											
ActionArgsTail					R24			R23								
SystemBlock																
StmtList			R27													
Stmt																
ForStmt																
IfStmt																
Condition																
CompOp									R35	R36	R37	R38	R39	R40		
Expr				R42		R42										
ArgList					R44											
ArgTail					R46			R45								
ArithExpr				R47		R47										
ArithTail			R50		R50											R50
Term				R51		R51										
TermTail			R54		R54											R54
Factor				R60		R61										
ListLit						R62										
ListItems							R64									
ListItemsTail							R66	R65								
Arg																
Type																

7. Lexer

Design Decisions for lexer:

• **Longest-prefix match:** Python's re.finditer() scans left-to-right and finds the longest match at each position. Multi-character operators (-> == != <= >=) are placed BEFORE their single-character prefixes to guarantee correct priority.

• **Keyword vs. Identifier:** All keywords are matched by the ID regex first (since they are valid identifiers), then a post-match dictionary lookup promotes them to their keyword token type.

• **Separate type keyword tokens:** The type keywords string/int/bool/list are given token types STRING_TYPE / INT_TYPE / BOOL_TYPE / LIST_TYPE so they never collide with STRING_LIT / INT_LIT in the grammar or parser.

• **Comments:** Single-line comments start with # and are skipped.

• **Line & column tracking:** The lexer tracks the byte offset of each newline so it can report accurate line and column numbers in error messages.

Token Table:

Complete list of token types produced by the lexer:

AGENT TOOL TASK ACTION SYSTEM FOR IN IF RUN

TRUE FALSE

STRING_TYPE INT_TYPE BOOL_TYPE LIST_TYPE

ID INT_LIT STRING_LIT

ARROW EQEQ NEQ LEQ GEQ LT GT EQ

PLUS MINUS MULT DIV

LBRACE RBRACE LPAREN RPAREN LBRACKET RBRACKET

COMMA COLON DOT

8. Parser

The parser (dsl_parser.py) is a Recursive Descent LL(1) parser.

Methods implemnted

• **peek(how_far=1):** Returns the token how_far positions ahead without consuming it.

• **get_token():** Consumes and returns the next token, advancing the index.

• **expect(token_type):** Calls get_token() and raises a SyntaxError if the returned type does not match.

• **parse_X() per non-terminal:** Each of the 35 non-terminals has a dedicated parse function. The function uses peek() to select the right production (LL(1) lookahead), then calls expect() or other parse functions to consume the RHS.

Example: parse_stmt():

```
t = self.peek().type
if t in TYPE_TOKENS:   # string / int / bool / list → VarDecl (R28)
    self.parse_type(); self.expect('ID'); self.expect('EQ'); self.parse_expr()
elif t == 'ID':        # Assignment (R29)
    self.expect('ID'); self.expect('EQ'); self.parse_expr()
elif t == 'FOR':       # ForStmt (R30)
    self.parse_for_stmt()
elif t == 'IF':        # IfStmt  (R31)
    self.parse_if_stmt()
else: self._syntax_error('expected statement', self.peek())
```

Note:

ArithTail and TermTail are implemented with while-loops rather than recursion to avoid deep call stacks on long arithmetic expressions. This is semantically equivalent to the right-recursive grammar rules R48–R54 but more practical in Python.

9. Example DSL Programs

Full example (example.dsl)

Demonstrates all features: agents, typed variables, for-loop, if-statement, run calls, arithmetic.

```
agent Researcher {
    tool web_search
    tool llm
    task gather(string topic) -> string data {
        action: web_search(topic)
        action: llm("summarize results")
    }
}

agent Analyzer {
    tool llm
    task sentiment(string text) -> string result {
        action: llm("detect sentiment")
    }
}

system {
    list topics = ["AI", "Robotics", "Security"]
    int i = 0
    bool negative_found = false

    for t in topics {
        string data    = run Researcher.gather(t)
        string opinion = run Analyzer.sentiment(data)
        if opinion == "negative" {
```

```
            negative_found = true
        }
        i = i + 1
    }
}
```

Minimal Example (Agent with no return Value)

```
agent Logger {
    tool llm
    task log(string msg) {
        action: llm("record entry")
    }
}
system {
    string entry = "start"
    entry = run Logger.log(entry)
}
```

Error Example:

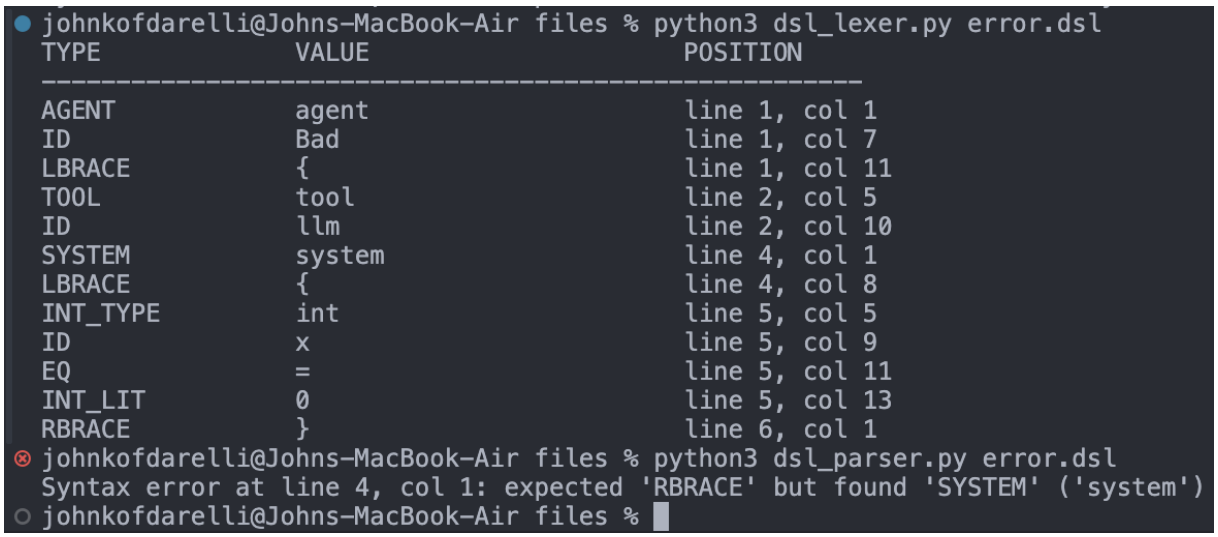
Input that should cause a syntax error (missing closing brace):

```
agent Bad {
    tool llm
    # missing closing brace → syntax error
system {
    int x = 0
}
```

Expected output: Syntax error at line 4, col 1: expected 'RBRACE' but found 'SYSTEM'

Testing:

We asked chatgpt to come up with a way to test our lexer and parser, and he came up with a simple command line entry point. Running it on our error example, we get the following output:



10. Design Decisions and Documentations

• **Grammar is purely LL(1):** Every parsing decision requires only one token of lookahead. We verified this by computing FIRST and FOLLOW sets and confirming there are no FIRST/FIRST or FIRST/FOLLOW conflicts.

• **No left recursion:** Arithmetic precedence is encoded through the standard chain ArithExpr → Term → Factor. Left-recursive rules like E → E + T were rewritten as right-recursive rules (R47–R54) to allow top-down parsing.

• **Separate Type and Literal token types:** The DSL type keywords (string, int, bool, list) are tokenised as STRING_TYPE / INT_TYPE / BOOL_TYPE / LIST_TYPE to ensure they never appear in the same FIRST set as string/integer literals.

• **Stmt alternatives are mutually exclusive:** VarDecl always starts with a type keyword; Assignment always starts with an ID; ForStmt starts with 'for'; IfStmt starts with 'if'. No ambiguity is possible.

• **Condition is left simple:** The condition in an if-statement is restricted to 'ID CompOp Arg'. This covers all examples in the project spec while keeping the grammar simple. More complex conditions (nested boolean expressions) can be added in Phase 2.

• **action keyword is a special keyword:** Inside a task body, each action line starts with the reserved word 'action' followed by a colon. This makes ActionStmt easy to recognise with one token of lookahead (FIRST = {action}).

• **Python is used because it is simple and readable:** Using Python keeps the parser easy to read and we already have a lot of the code from the lexer covered in class.